

PROJECT 03 · WEB APPLICATION SECURITY

SQL Injection Vulnerability Assessment

Target: PortSwigger Web Security Academy — Intentionally Vulnerable Lab

Vulnerability: SQL Injection

Affected Function: Product Stock Check

Affected Parameter: storeId

Endpoint: POST /product/stock

Testing Tool: Burp Suite

Assessment Type: Web Application Security Assessment

Tester: Lesiba Simon Mahlatse Legodi

Date: August 2026

1. Executive Summary

A security assessment was performed against an intentionally vulnerable web application provided by PortSwigger Web Security Academy.

The assessment identified a **SQL Injection vulnerability** in the storeId parameter of the product stock-check functionality. The parameter was found to accept attacker-controlled input that was subsequently interpreted as part of a database query.

The vulnerability was validated by supplying SQL expressions and observing changes in the application's response. An initial UNION SELECT attempt was blocked by a Web Application Firewall (WAF). XML entity encoding was subsequently used within the authorized laboratory environment to demonstrate that the filtering mechanism could be bypassed.

Further testing established that the underlying query returned a single column. A SQL query was then constructed to concatenate usernames and passwords from the application's users table into the available output column.

The assessment successfully demonstrated unauthorized retrieval of user credentials and authentication as the administrator account.

This demonstrates a significant confidentiality and authentication risk if a similar vulnerability existed in a real production application.

2. Scope and Authorization

Testing was conducted exclusively against the intentionally vulnerable PortSwigger Web Security Academy laboratory environment.

No real-world websites, systems, applications, databases, or third-party infrastructure were targeted.

The assessment was limited to the functionality provided within the selected laboratory.

In-scope component:

- Product stock-check functionality
- POST /product/stock
- storeId parameter
- Database interaction associated with the stock-check request

Out of scope:

- Real production systems
- Third-party websites
- Unrelated PortSwigger laboratories
- Systems not explicitly provided for the exercise

3. Vulnerability Overview

3.1 What is SQL Injection?

SQL Injection is a web application vulnerability that occurs when an application incorporates untrusted user input into a database query without adequately separating data from SQL instructions.

An attacker may manipulate the application's input so that the database interprets part of the supplied input as SQL syntax.

Depending on the application's functionality and database permissions, SQL Injection may allow an attacker to:

- Read unauthorized database information
- Modify database records

- Delete data
- Bypass authentication
- Access sensitive credentials
- Potentially execute further database operations

In this assessment, SQL Injection was identified in the storeId parameter of the stock-check request.

4. Environment and Tools

The assessment used the following:

Tool	Purpose
PortSwigger Web Security Academy	Intentionally vulnerable testing environment
Burp Suite	HTTP interception and request manipulation
Burp Repeater	Repeated testing and modification of HTTP requests
Hackvertor	XML entity encoding for the laboratory WAF-bypass demonstration
Web Browser	Accessing the laboratory application

5. Methodology

The assessment followed these stages:

1. Establish a baseline request.
2. Identify the storeId parameter.
3. Test whether the parameter was evaluated.
4. Test for SQL injection.
5. Identify the behavior of the WAF.
6. Demonstrate the authorized WAF bypass.
7. Determine the number of columns returned by the query.
8. Construct a single-column data extraction query.
9. Retrieve database credentials.

10. Demonstrate authentication impact.
11. Document remediation recommendations.

6. Vulnerability Identification

Step 1 — Identify the Stock Check Request

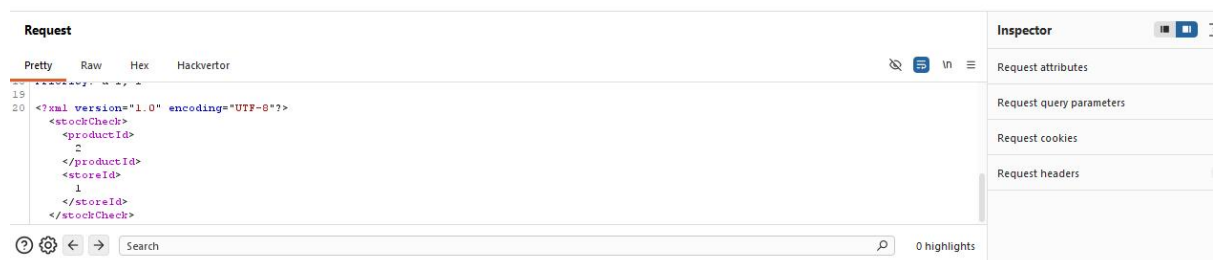
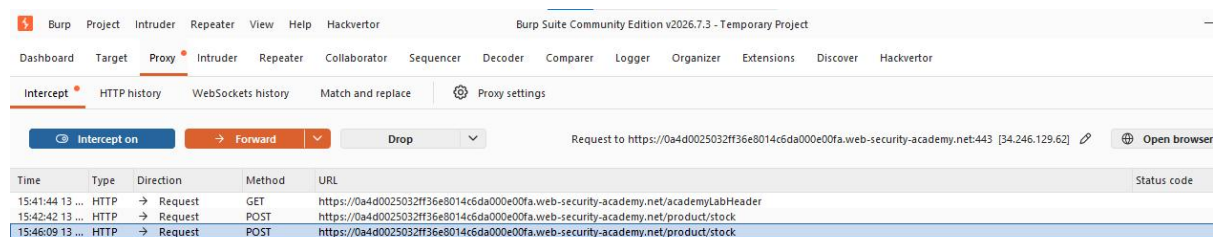
The application provides a stock-check functionality that allows a user to determine whether a product is available at a particular store.

When the stock check is submitted, the application sends the productId and storeId to the server in XML format.

The request was intercepted using Burp Suite.

Evidence

Screenshot 01 — Original Stock Check Request



The relevant portion of the request was:

```
<storeId>1</storeId>
```

The storeId value was selected as the primary injection point because it was supplied by the client and subsequently processed by the server.

7. Testing Whether the Parameter Is Evaluated

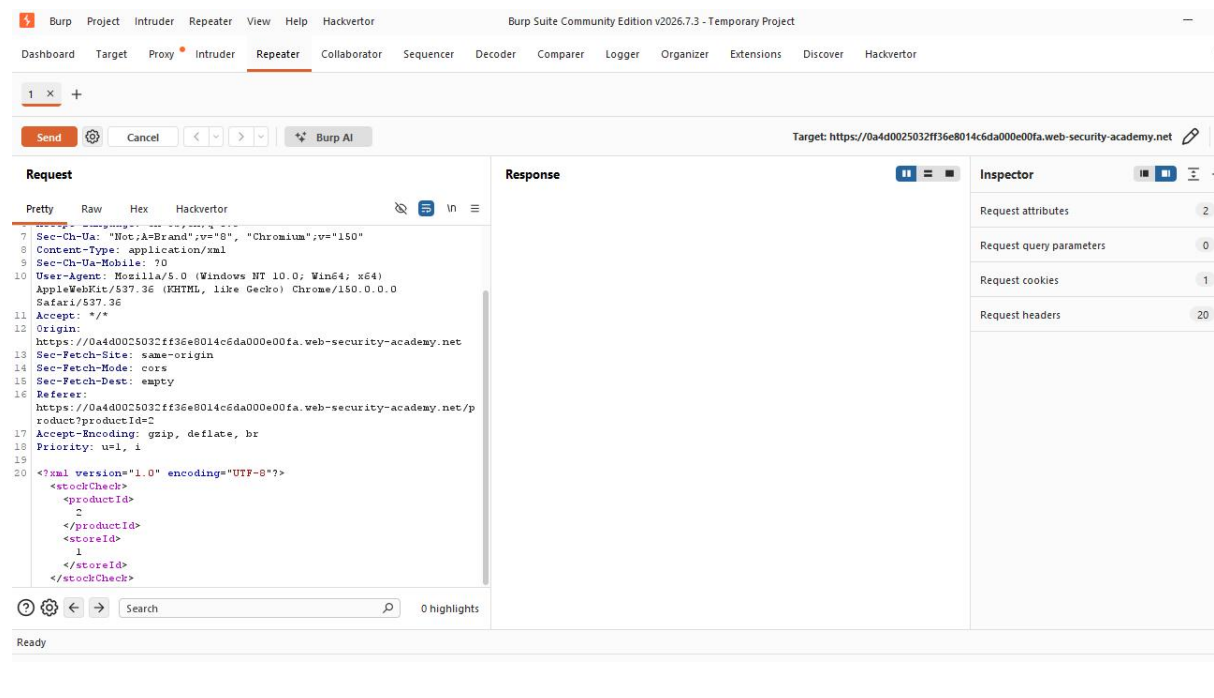
Step 2 — Send the Request to Burp Repeater

The stock-check request was sent from Burp Proxy to Burp Repeater.

Burp Repeater was used because it allows the same request to be repeatedly modified and submitted while comparing application responses.

Evidence

Screenshot 02 — Request in Burp Repeater



Step 3 — Submit a Mathematical Expression

The original storeId value was modified from:

```
<storeId>1</storeId>
```

to:

```
<storeId>1+1</storeId>
```

The modified request was then submitted.

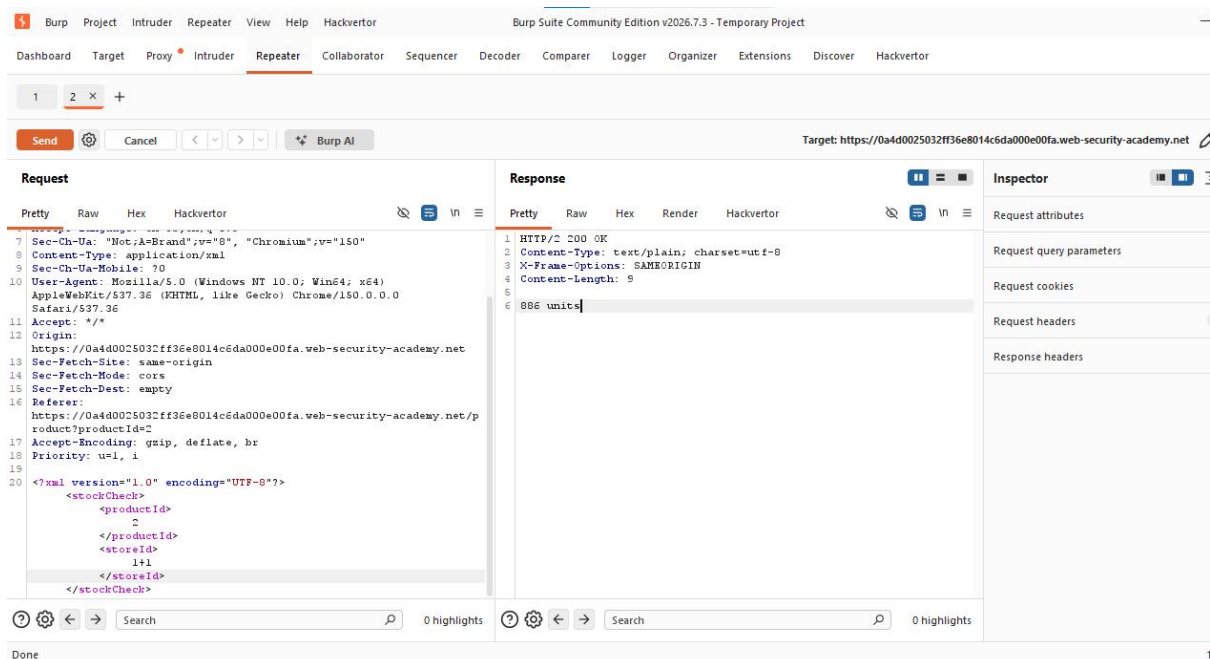
Observation

The application returned stock information corresponding to a different store.

This indicated that the supplied expression was being evaluated rather than simply being treated as a literal store identifier.

Evidence

Screenshot 03 — Mathematical Expression Test



Interpretation

The behavior provided an initial indication that the storeId parameter could influence the underlying database operation.

Further testing was therefore performed to determine whether the parameter was vulnerable to SQL Injection.

8. Confirming SQL Injection

Step 4 — Test a UNION SELECT Statement

A UNION SELECT statement was introduced into the storeId parameter.

The laboratory exercise used:

1 UNION SELECT NULL

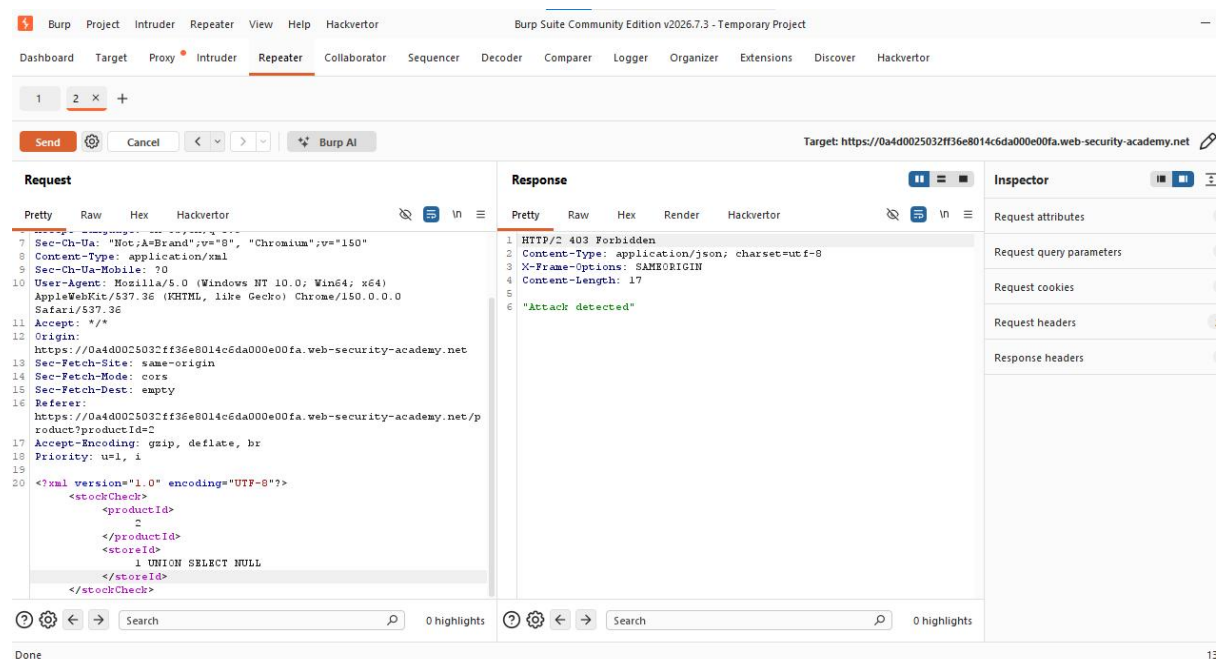
The resulting XML parameter therefore contained the SQL expression within the storeId element.

Observation

Instead of receiving the expected application response, the request was blocked and identified as a potential attack.

Evidence

Screenshot 04 — Initial UNION SELECT Blocked



Interpretation

The response indicated that a security filtering mechanism, such as a Web Application Firewall, was detecting the SQL injection syntax.

The blocked request did not prove that the underlying SQL Injection was absent. Instead, it demonstrated that an additional security control was interfering with the test.

9. WAF Bypass Using XML Entity Encoding

Step 5 — Encode the SQL Payload

Because the injection was being supplied inside an XML request, XML entity encoding was investigated.

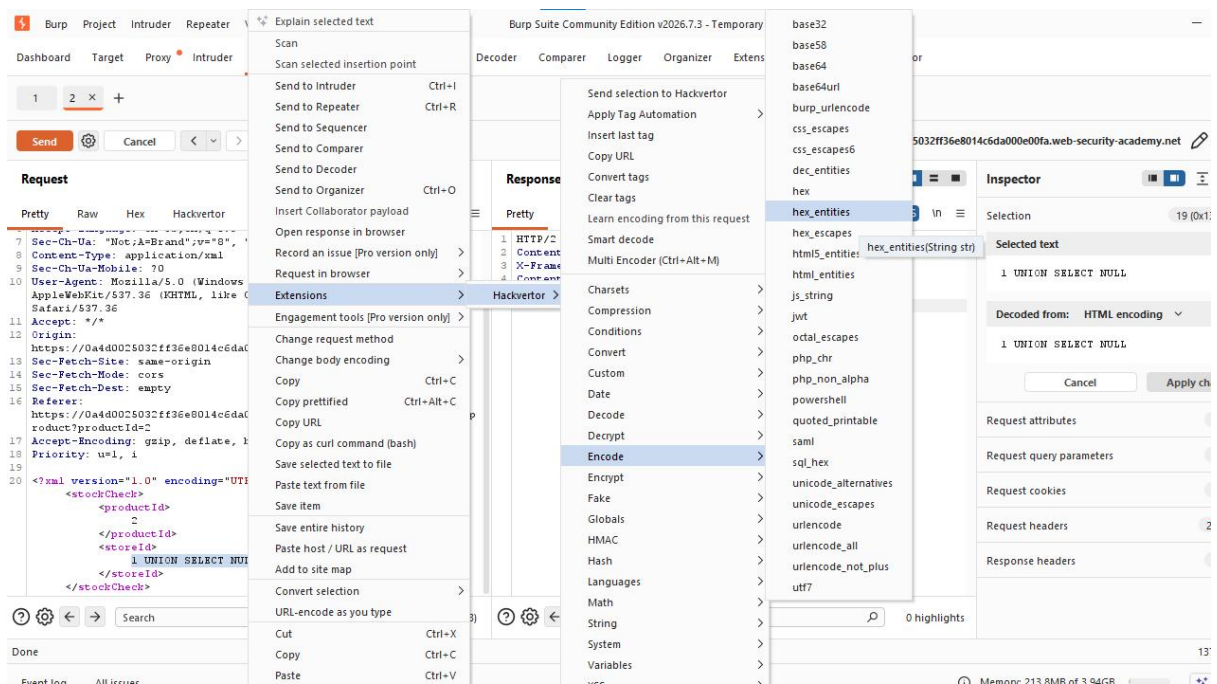
Hackvertor was used within Burp Suite to encode the SQL injection payload.

The relevant encoding functionality was:

****Extensions → Hackvertor → Encode → **hex_entities**

Evidence

Screenshot 05 — Hackvertor Encoding



The encoded representation was then inserted into the XML request.

Step 6 — Resend the Encoded Request

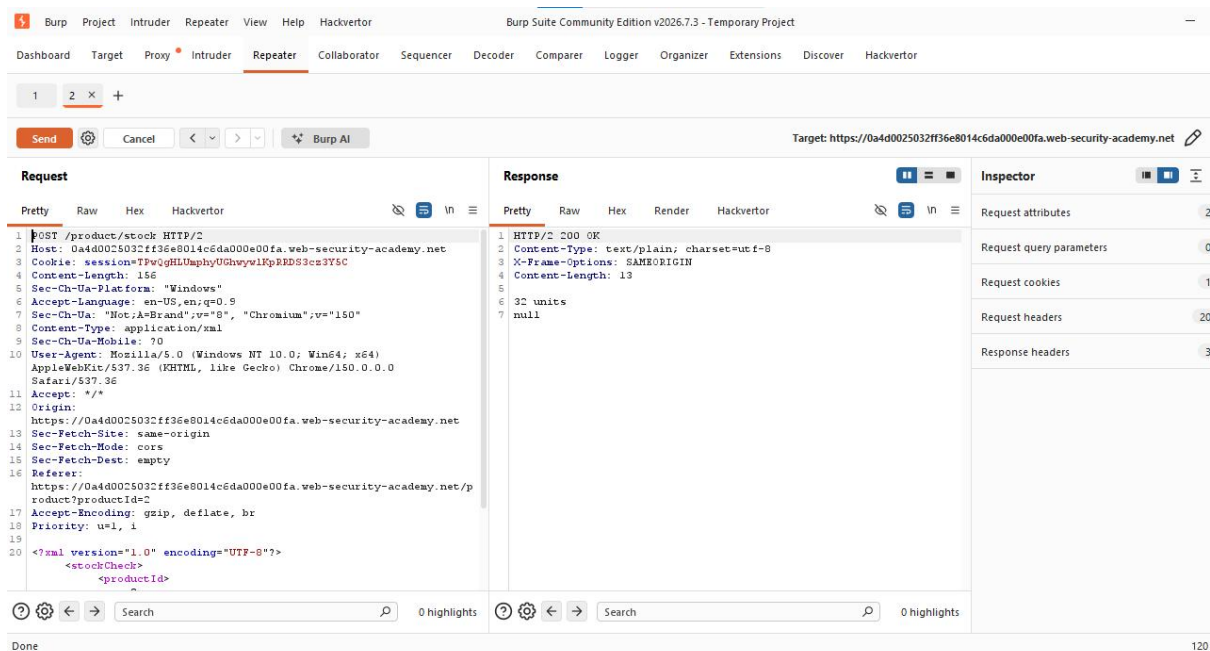
The encoded request was submitted through Burp Repeater.

Observation

The previously blocked request now produced a normal application response.

Evidence

Screenshot 06 — Encoded Payload Accepted



Interpretation

This demonstrated that the WAF's filtering mechanism could be bypassed within the controlled laboratory environment.

The important security issue remained the underlying SQL Injection vulnerability. The WAF bypass demonstrated that the existing filtering mechanism was not a reliable primary defense against SQL Injection.

10. Determining the Number of Columns

Step 7 — Test the Query Structure

The next objective was to determine how many columns were returned by the underlying SQL query.

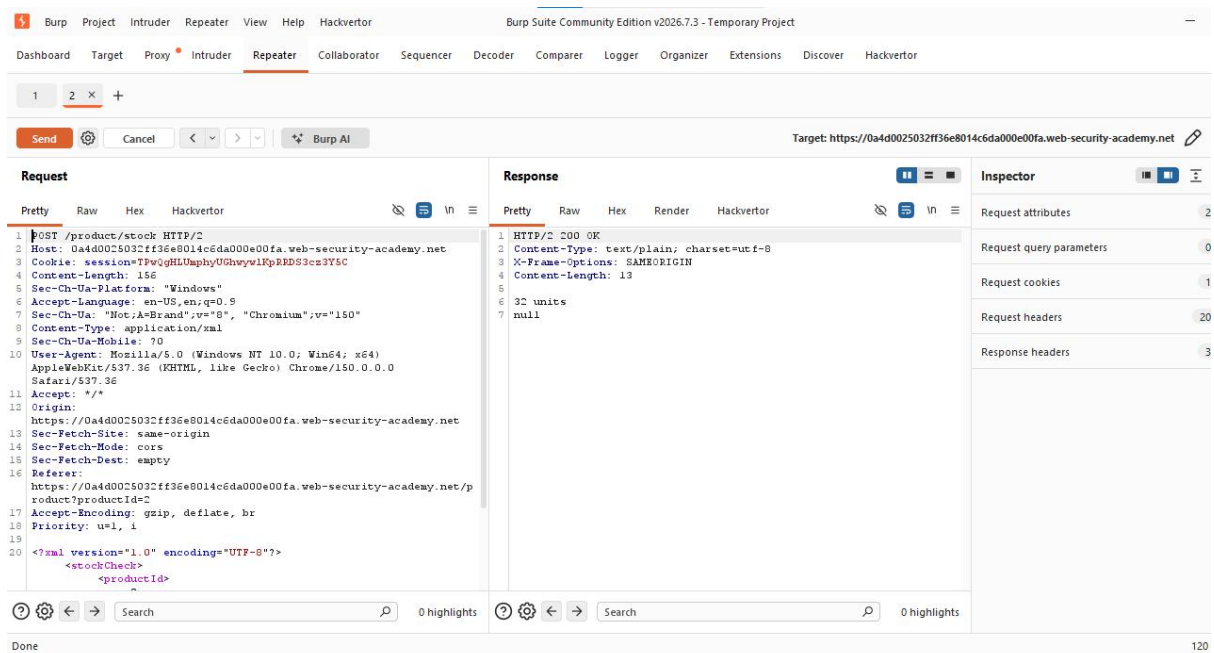
A UNION SELECT statement was used with a single NULL value.

The response indicated that the query accepted a single returned column.

Additional columns resulted in an error-like response or 0 units.

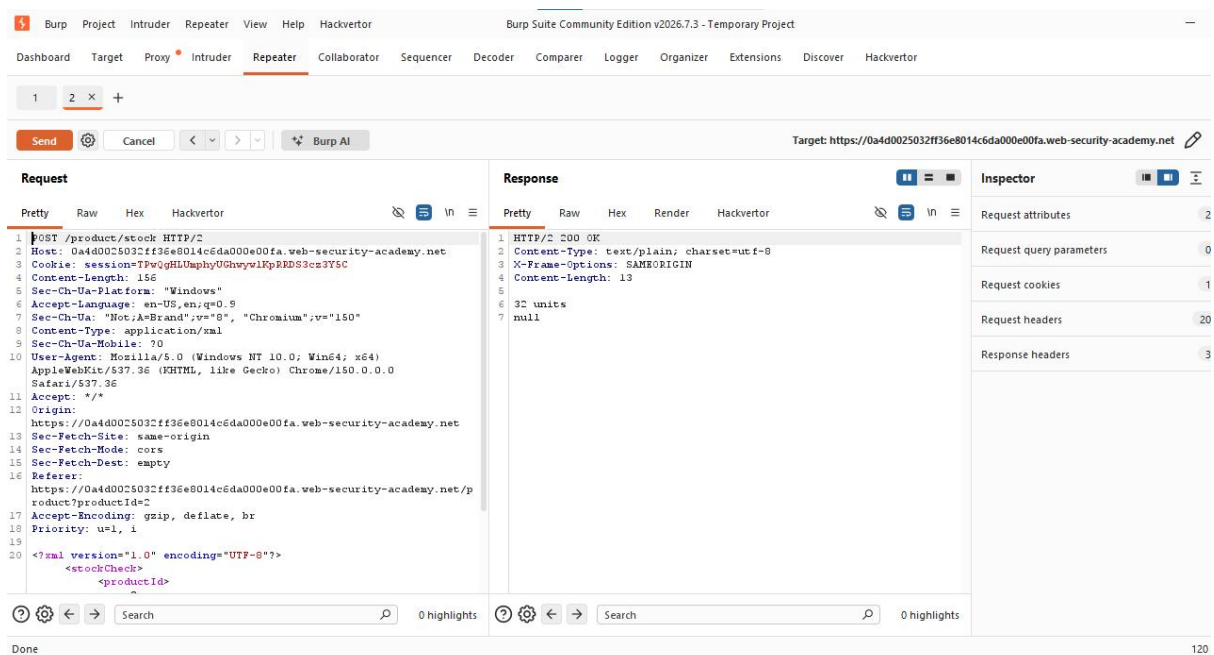
Evidence

Screenshot 07 — Single-Column UNION Test



Evidence

Screenshot 08 — Multiple-Column Test



Finding

The behavior indicated that the original query returned **one column**.

This was important because a UNION SELECT query must have a compatible number of columns with the original query.

11. Constructing the Data Extraction Payload

Step 8 — Address the Single-Column Limitation

The application allowed only one column to be returned.

However, both the username and password needed to be displayed for the laboratory demonstration.

The values were therefore concatenated into a single output value using the database's string-concatenation operator.

A delimiter was placed between the username and password so that the two values could be distinguished.

The laboratory payload used the following structure:

```
username || '~' || password
```

This produces output conceptually similar to:

```
username~password
```

The query was then combined with the users table.

12. Extracting Database Credentials

Step 9 — Submit the Final Laboratory Payload

The SQL expression was encoded using the XML entity encoding technique demonstrated earlier and submitted through the storeId parameter.

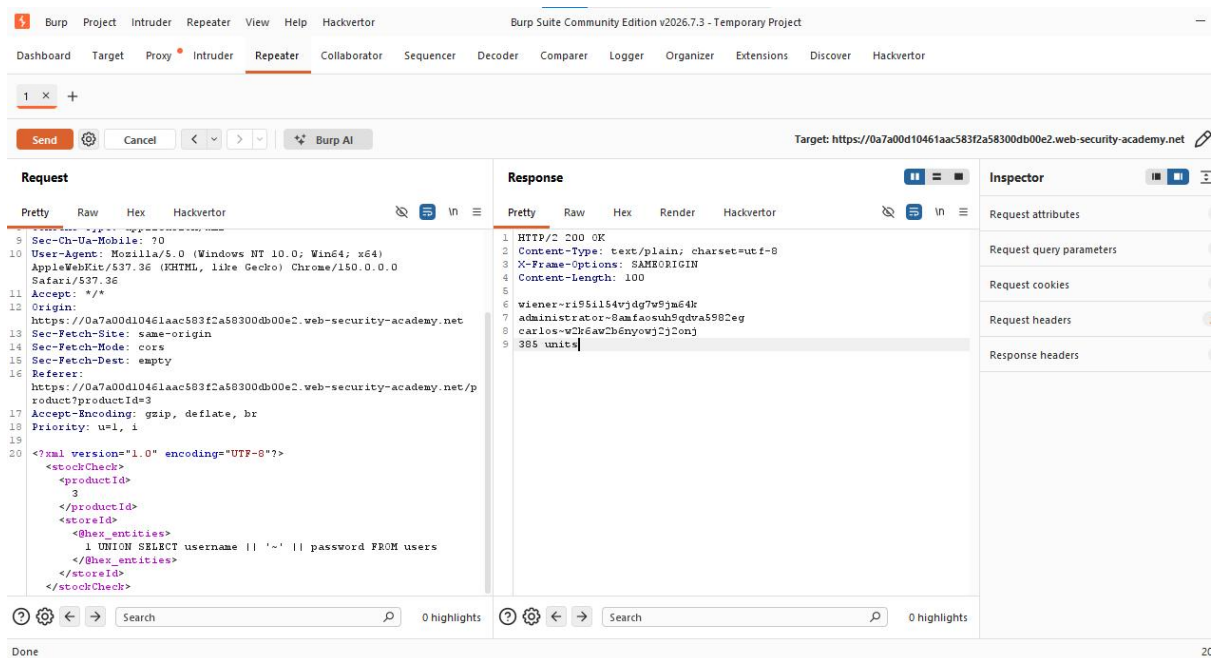
The laboratory query was designed to return:

- Username
- ~ delimiter
- Password

from the users table.

Evidence

Screenshot 09 — Credential Extraction



Observation

The application returned database records containing usernames and passwords.

The administrator account was identified among the returned records.

Security Impact

This demonstrated that the SQL Injection vulnerability could be used to retrieve sensitive authentication information from the application's database.

13. Demonstrating Authentication Impact

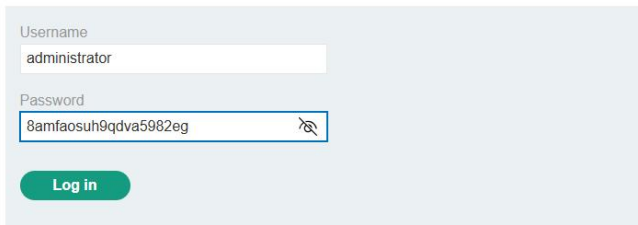
Step 10 — Authenticate as the Administrator

The administrator credentials obtained during the authorized laboratory exercise were used to authenticate to the application.

Evidence

Screenshot 10 — Administrator Authentication

Login



Username
administrator

Password
8amfaosuh9qdv5982eg 

Log in

Observation

Authentication as the administrator account was successful.

Impact Demonstrated

The vulnerability therefore extended beyond database information disclosure.

The assessment demonstrated a potential path from:

SQL Injection → Database Access → Credential Disclosure → Account Compromise

This provides strong evidence of the practical security impact of the vulnerability.

14. Proof of Concept

14.1 *Reproduction Steps*

An authorized tester can reproduce the vulnerability within the laboratory using the following process:

1. Access the PortSwigger SQL Injection laboratory.
2. Open the product stock-check functionality.
3. Intercept the stock-check request using Burp Suite.
4. Identify the XML storeId parameter.
5. Send the request to Burp Repeater.
6. Modify the storeId value with a mathematical expression.

7. Observe the change in application behavior.
8. Test a UNION SELECT expression.
9. Observe the WAF response.
10. Encode the SQL expression using XML entities.
11. Resend the encoded request.
12. Determine that the underlying query returns one column.
13. Concatenate username and password values into the single available column.
14. Retrieve the database records.
15. Identify the administrator account.
16. Demonstrate authentication using the retrieved laboratory credentials.

15. Expected Result vs Actual Result

Test	Expected Result	Actual Result
Normal storeId	Stock information for requested store	Stock information returned
Mathematical expression	Input treated as ordinary ID	Expression affected application behavior
Initial UNION query	Application should reject invalid store ID	WAF blocked the request
Encoded SQL expression	Security control should still prevent injection	Request received a normal application response
Single-column UNION	Compatible query should return data	Single-column behavior confirmed
Credential extraction	No unauthorized database information	Username and passwords returned
Administrator login	Only authorized credentials should authenticate	Administrator authentication succeeded

16. Vulnerability Root Cause

The underlying root cause is insufficient separation between application data and SQL instructions.

The application allows attacker-controlled input from the stored parameter to influence a database query.

Instead of treating the supplied value strictly as data, the database interprets portions of the input as SQL syntax.

This creates the possibility of query manipulation.

The WAF provides an additional security layer, but the assessment demonstrated that filtering alone is insufficient to prevent the underlying vulnerability.

A secure application should prevent SQL Injection at the application/database interaction layer rather than relying primarily on a WAF to identify malicious strings.

17. Security Impact

The demonstrated impact includes:

Confidentiality

Sensitive database information, including usernames and passwords, could be retrieved.

Authentication

Retrieved credentials could be used to authenticate as another user.

Account Compromise

The laboratory demonstration successfully resulted in authentication as the administrator.

Potential Broader Impact

In a real application, the ultimate impact would depend on database privileges, application functionality, exposed information, and the privileges associated with compromised accounts.

Potential consequences could include:

- Unauthorized access to sensitive information
- Account takeover
- Exposure of customer or employee data
- Unauthorized modification of application data
- Further compromise of application functionality

The exact impact should be assessed according to the privileges available to the vulnerable database connection and affected accounts.

18. Risk Assessment

Suggested Severity: High

The vulnerability should be considered **High severity** for this assessment because exploitation allows unauthorized database information disclosure and demonstrated administrator account compromise.

The final severity in a real-world assessment should consider factors such as:

- Whether authentication is required
- Database privileges
- Sensitivity of stored information
- Whether credentials are securely hashed
- Application privileges
- Network exposure
- Business impact

The PortSwigger laboratory provides a controlled demonstration of a high-impact outcome.

19. Remediation Recommendations

19.1 *Use Parameterized Queries*

The primary remediation is to use parameterized queries or prepared statements.

Application code should ensure that user-controlled values are passed as data parameters rather than concatenated directly into SQL statements.

Conceptually:

```
SELECT ... FROM ... WHERE store_id = ?
```

The value supplied by the user should then be bound to the parameter separately.

19.2 Avoid Dynamic SQL Construction

Applications should avoid constructing SQL statements by directly concatenating user-controlled input.

Unsafe architecture:

SQL query + user input

Preferred architecture:

Parameterized SQL query + bound parameter

19.3 Implement Server-Side Input Validation

The application should validate storeId according to the expected data type and business rules.

For example, if a store identifier is expected to be an integer, the server should validate that the supplied value conforms to the expected format.

Validation should be treated as an additional security measure rather than the primary defense against SQL Injection.

19.4 Apply Least-Privilege Database Access

The application's database account should have only the permissions required for normal application functionality.

The application should not use an unnecessarily privileged database account.

This reduces the potential impact if SQL Injection occurs.

19.5 Do Not Store Passwords in Plaintext

The assessment demonstrated retrieval of username/password values from the laboratory database.

Production applications should never store user passwords in plaintext.

Passwords should be stored using an appropriate password hashing algorithm with a unique salt, such as Argon2id, bcrypt, or an appropriately configured PBKDF2 implementation.

19.6 Improve WAF Configuration

The WAF should continue to be used as a defense-in-depth mechanism.

However, it should not be considered a replacement for secure application coding.

WAF rules should be regularly tested against encoding, normalization, and obfuscation techniques.

19.7 Implement Secure Error Handling

Database errors and internal implementation details should not be exposed to users.

Applications should return generic error messages while recording detailed information securely in internal logs.

19.8 Conduct Security Testing

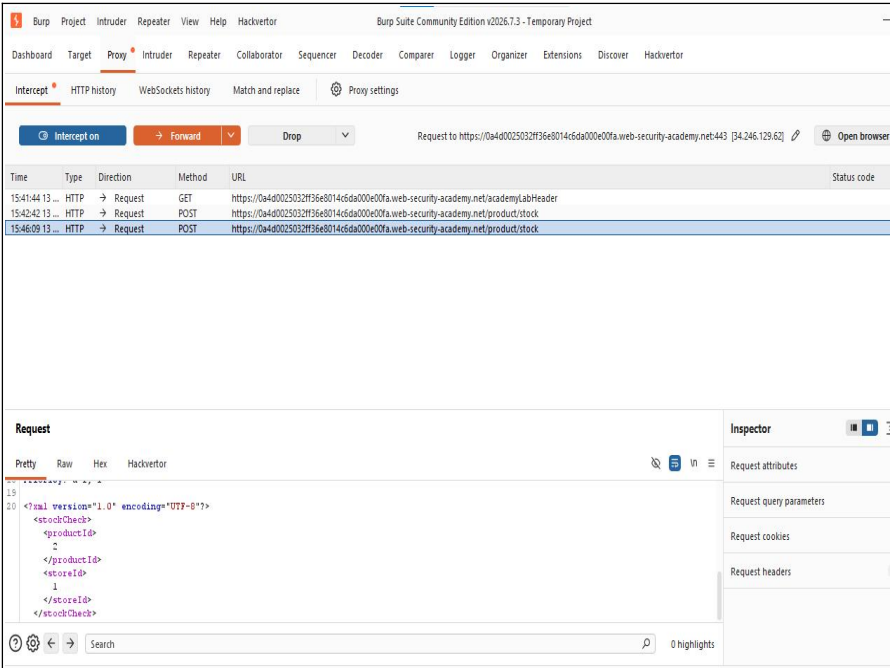
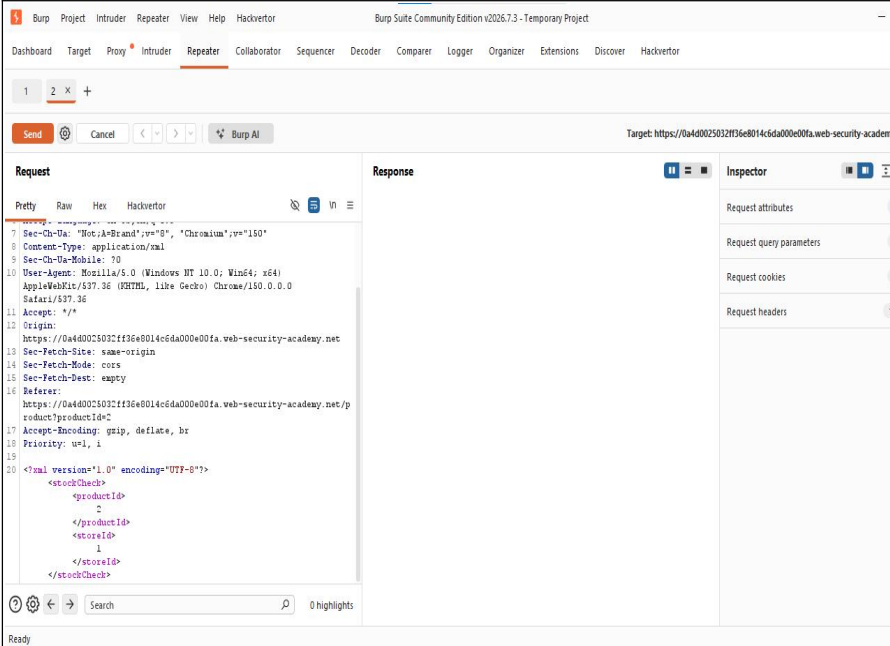
The application should undergo regular:

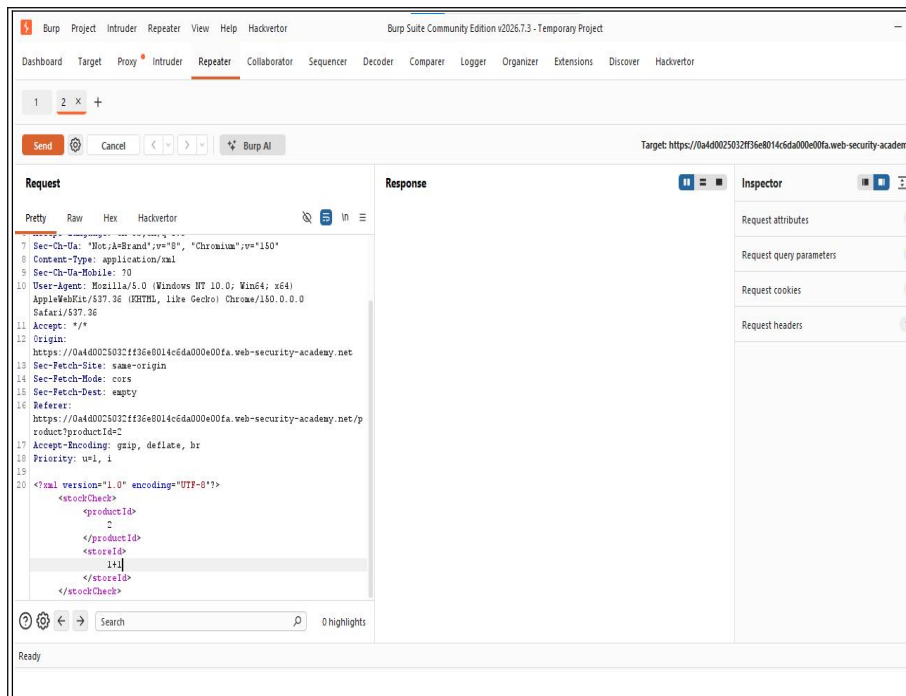
- Secure code reviews
- SAST
- DAST
- Dependency scanning
- Penetration testing
- SQL Injection testing

- Regression testing

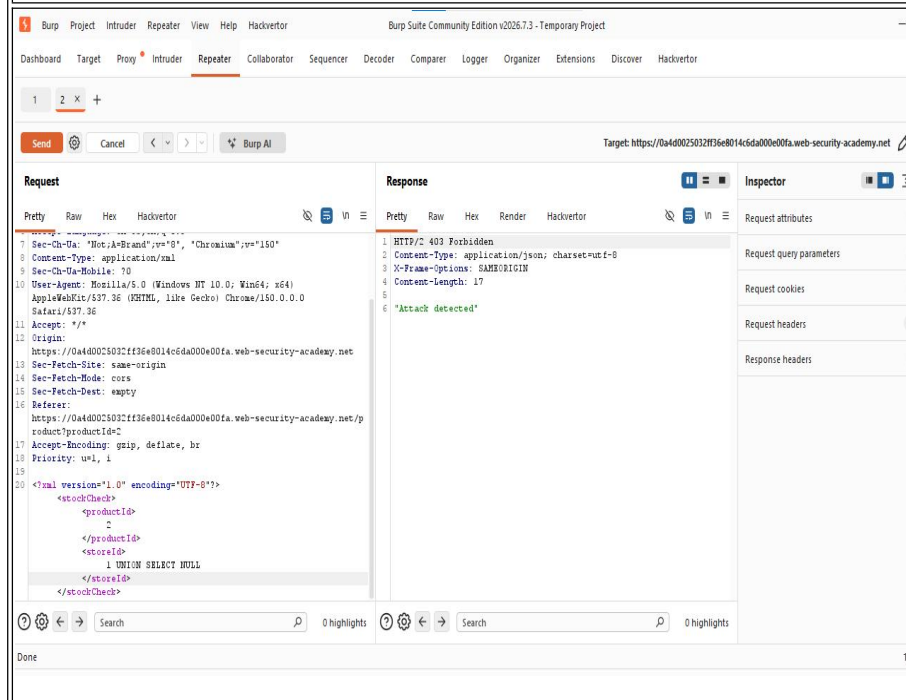
Security testing should occur during development and before production deployment.

20. Evidence Register

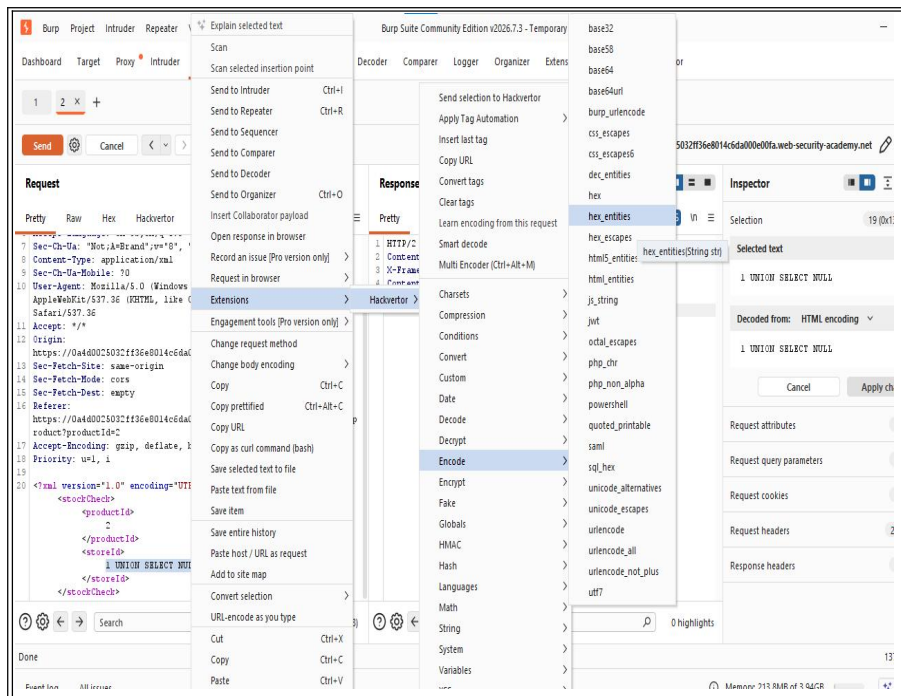
Evidence ID	Description
 The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. The 'HTTP history' pane displays a list of requests. The selected request is a POST to <code>https://0a40025032ff36e8014c6da000e00fa.web-security-academy.net/product/stock</code>. The 'Request' pane shows the raw HTTP request body as an XML document with a root element <code><stockCheck></code> containing <code><productId>2</productId></code> and <code><storeId>1</storeId></code>.	Original stock-check request
 The screenshot shows the Burp Suite 'Repeater' tab. A request has been transferred from the HTTP history. The 'Request' pane shows the raw HTTP request, including headers like <code>Sec-Ch-Ua</code>, <code>Content-Type</code>, <code>User-Agent</code>, and <code>Accept-Encoding</code>. The request body is the same XML as in the previous screenshot. The 'Response' pane is currently empty.	Request transferred to Burp Repeater



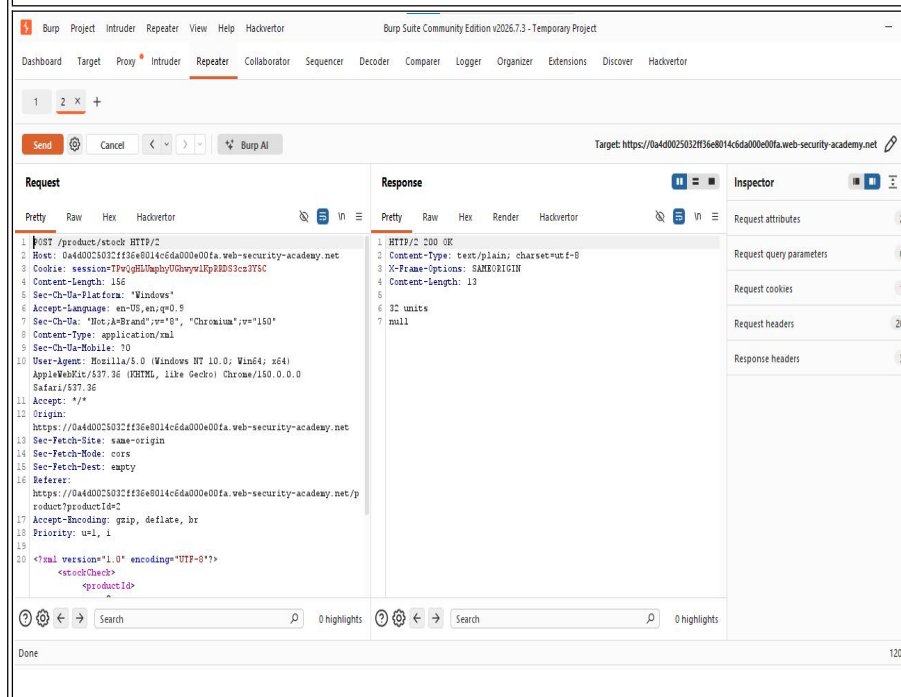
Mathematical
expression test



Initial UNION
SELECT blocked



Hackvortor XML
entity encoding



Encoded request
accepted

1 2 X +

Send Cancel < > Burp AI

Target: https://0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net

Request

Pretty Raw Hex Hackvortor

```
1 POST /product/stock HTTP/2
2 Host: 0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net
3 Cookie: session=TPwqgLLUghy0QhmywLPpXZDS3cs375C
4 Content-Length: 156
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not.A=Brand";v="0", "Chromium";v="150"
8 Content-Type: application/xml
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net/product?productId=2
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 <?xml version="1.0" encoding="UTF-8">
  <stockCheck>
    <productId>
```

Response

Pretty Raw Hex Render Hackvortor

```
1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 13
5
6 32 units
7 null
```

Inspector

Request attributes

Request query parameters

Request cookies

Request headers

Response headers

Done 120

Single-column
UNION test

1 2 X +

Send Cancel < > Burp AI

Target: https://0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net

Request

Pretty Raw Hex Hackvortor

```
1 POST /product/stock HTTP/2
2 Host: 0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net
3 Cookie: session=TPwqgLLUghy0QhmywLPpXZDS3cs375C
4 Content-Length: 156
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not.A=Brand";v="0", "Chromium";v="150"
8 Content-Type: application/xml
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0a4d0025032ff36e8014c6da00e00fa.web-security-academy.net/product?productId=2
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 <?xml version="1.0" encoding="UTF-8">
  <stockCheck>
    <productId>
```

Response

Pretty Raw Hex Render Hackvortor

```
1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 13
5
6 32 units
7 null
```

Inspector

Request attributes

Request query parameters

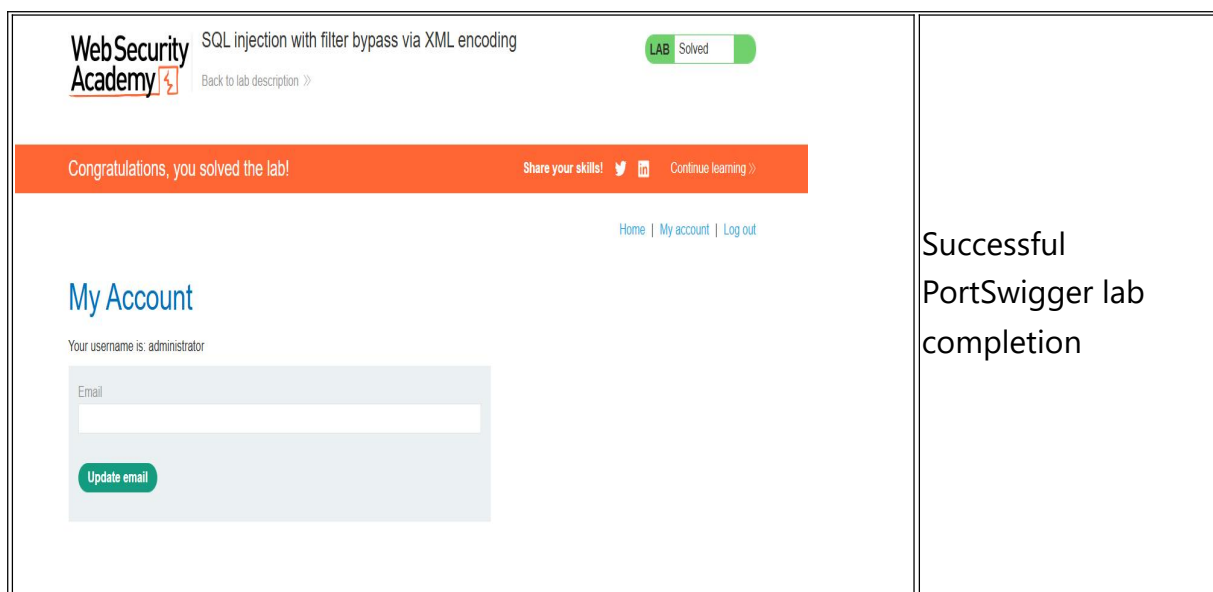
Request cookies

Request headers

Response headers

Done 120

Multiple-column test



21. Conclusion

The assessment successfully identified and demonstrated a SQL Injection vulnerability within the product stock-check functionality of the intentionally vulnerable PortSwigger Web Security Academy laboratory.

The vulnerable storeId parameter was shown to influence the application's database query. Subsequent testing demonstrated that SQL syntax could be introduced into the query and that the initial WAF protection could be bypassed using XML entity encoding.

Testing established that the underlying query returned a single column, after which username and password values were concatenated and retrieved from the application's users table.

The extracted administrator credentials were subsequently used to authenticate successfully, demonstrating the practical impact of the vulnerability.

The primary remediation is to eliminate SQL Injection at the application layer through parameterized queries or prepared statements. Additional controls, including input validation, least-privilege database permissions, secure password storage, appropriate error handling, and WAF protection, should be implemented as defense-in-depth measures.

The assessment demonstrates that relying solely on input filtering or WAF rules is insufficient. Secure database interaction must be implemented within the application itself.

22. **Final Assessment Statement**

Finding: SQL Injection in the product stock-check storeId parameter.

Severity: High

Primary Risk: Unauthorized database access and credential disclosure.

Demonstrated Impact: Retrieval of usernames and passwords followed by successful administrator authentication within the authorized laboratory environment.

Primary Recommendation: Replace dynamically constructed SQL queries with parameterized queries/prepared statements and enforce least-privilege database access.